

Comparing Quantization Techniques on Image Classification

Bo Ding J. Keith Hall Dustin Newman Schuyler R. Wang
Georgia Institute of Technology, Atlanta, GA

Abstract

Quantization is an increasingly popular set of techniques to reduce the size of deep neural networks targeting resource-constrained or edge environments. We evaluate the effects of both uniform and mixed-precision quantization on a standard ResNet-50 image classifier trained on the CIFAR-100 dataset, comparing post-training quantization (PTQ) and quantization-aware training (QAT). Uniform 8-bit PTQ is effectively lossless (79.7% vs. 79.8% top-1) but collapses to near-chance at 4-bit (~ 7 –8% top-1), whereas fold-aware QAT recovers 4-bit accuracy to 66.9–75.7% at the same $7.94\times$ compression, a difference of up to 67 points, at the cost of a ~ 2.7 -point retraining penalty that leaves it behind PTQ at 8-bit. Mixed-precision allocation traces the accuracy-per-bit frontier between these regimes. We further compare PTQ and fold-aware QAT using identical HAWQ-V2 mixed-precision allocations.

1. Introduction

As the size and applications of deep neural networks (DNNs) grow, there is increased demand for methods to deploy large state-of-the-art DNN models on edge hardware and other devices bottlenecked by memory, compute, or energy constraints. DNN quantization is the process of compressing models by converting their weights or activations to low- or ultralow-precision integers (e.g., 8-bit or lower). Two important quantization paradigms that seek to shrink memory footprints without unacceptable accuracy degradation are Post-Training Quantization (PTQ) (after training) and Quantization-Aware Training (QAT) (during training), each of which can be implemented with uniform (all layers having the same compression) or mixed precision. In practice, uniform 8-bit integer (INT8) weight PTQ often results in lossless compression relative to 32-bit floating point (FP32) baselines (7), while performant compression into INT4 and beyond remains the frontier of current research. Mixed-precision quantization techniques seek to efficiently search the space of nonuniform quantization calibrations to find which weight or activation tensors are most or least sensitive to compression, thereby identifying where

one can obtain maximal memory savings for the smallest performance cost.

In this report, we compare four quantization techniques to determine which achieves the least accuracy degradation relative to compression ratio as applied to the ResNet-50 image classification model (5) on CIFAR-100 (8). ResNet-50 is a popular deep residual CNN whose stacked convolutional blocks build translation-equivariant hierarchical features and whose residual connections make its deep layers optimizable. CIFAR-100 is a benchmark dataset comprising 60k color images in 100 classes widely used in computer vision research to evaluate image classifiers. Inputs are $3 \times 32 \times 32$ tensors normalized by CIFAR-100 channel statistics with standard crop and flip augmentation; outputs are 100-dimensional logits mapped by softmax to a top-1 argmax or the five largest logits for top-5. The four techniques evaluated are uniform PTQ, fold-aware uniform QAT, and two mixed-precision allocators: HAWQ-V2 (3), which allocates bit-widths from Hessian-trace sensitivity, and EdMIPS (2), which learns them by differentiable search. We first show that INT8 weight PTQ is essentially lossless, whereas per-tensor INT4 PTQ collapses on folded ResNet weights. We then demonstrate that fold-aware QAT recovers most of the lost INT4 accuracy, and establish per-channel quantization as an effective strategy at lower precisions. Finally, we evaluate Hessian-guided HAWQ-V2 mixed-precision allocations under both PTQ and fold-aware QAT, enabling a direct comparison under identical bit allocations. Accuracy is reported against both compression ratio and analytic bit-operations (BOPS) to compare methods across common efficiency metrics.

2. Approach

Quantization requires calibrating the mapping from the original distribution of values to target values. The simplest approach maps each floating point representation of a source value x into one of $2^8 = 256$ equally spaced buckets in the integer range $[q_{\min}, q_{\max}]$ using

$$q = \text{clamp}(\text{round}(x/s) + z, q_{\min}, q_{\max})$$

where the scale factor s is a mapping of the source floating point range $[x_{\min}, x_{\max}]$ to the integer range, and the zero

point z is chosen so that floating point 0 maps exactly onto integer 0. Dequantizing back to floating point via $\hat{x} = (q - z) \cdot s$ allows us to measure the quantization error for a single value, which is bounded as $|x - \hat{x}| \leq s/2$. It follows that the presence of outliers in the source distribution can cause the scale factor to grow, resulting in an inefficient allocation of quantization buckets near the outlier at the expense of fewer buckets near the center of the distribution.

In this report, we evaluate both per-tensor and per-channel quantization techniques. Per-tensor quantization has only a single scale factor per weight tensor, whereas per-channel quantization stores a single scale factor per kernel. This is relevant after BatchNorm in particular, which scales each output channel independently, producing scale variations across channels (7). We calibrate weights by their minimum and maximum, and activations by a high percentile (e.g., 99.99%), as activations have heavy tails whose outliers would otherwise inflate the scale factor and waste quantization levels, whereas weight distributions have low to moderate kurtosis. Calculating activation percentiles across tensors with tens of millions of elements is impractical at every calibration step, so we estimate it from a fixed random subsample (on the order of 10^6 elements); a fixed seed keeps the estimate deterministic across runs. Calibration statistics are collected over a fixed set of 1,000 training images, and BatchNorm is folded into the preceding convolutions and verified to reproduce the unfolded network’s outputs to within 10^{-5} before any quantization is applied.

2.1. Fold-Aware QAT

We train QAT models from scratch under the identical recipe as the FP32 baseline for fair comparison. Rounding is made differentiable with a Straight-Through Estimator (STE). One difficulty specific to ResNet models is BatchNorm folding: at inference, each convolution–BatchNorm pair collapses to a single affine convolution with folded weight $\tilde{W} = \gamma W / \sqrt{\sigma^2 + \epsilon}$ and a folded bias (6). PTQ folds once on a converged model, but during QAT the BatchNorm running statistics are still moving, so the folded weights the quantizer must represent are a moving target.

To address this, we adopt a *freeze-then-fold* schedule: BatchNorm runs live (statistics updating, unfolded) for the first 150 of 200 epochs; thereafter, the statistics are frozen, the layers are folded, and the network trains on the folded, weight-quantized convolutions. Quantizing the folded weights during these final epochs lets the optimizer push them toward being representable at the target bit-width. A single fused layer implements both phases, running the convolution + BatchNorm before the freeze and the folded, weight-quantized convolution after. Weights are quantized throughout training, while activation quantization is calibrated and applied at evaluation, so each trained checkpoint yields two deployment points: weights-only and

weights-plus-activation.

Freezing BatchNorm mid-training introduces two instabilities we had to address. First, when training in FP16 for speed, the fold factor $\gamma / \sqrt{\sigma^2 + \epsilon}$ overflows for low-variance channels, producing non-finite folded weights and hence a non-finite loss; we therefore compute the folded path in full FP32 precision while the rest of the network runs in FP16. Second, freezing BatchNorm removes its implicit gradient normalization, so the unchanged SGD schedule diverges at the freeze point. Rather than dropping the learning rate, which would break schedule comparability with the baseline and PTQ arms, we clip gradients to a norm of 1.0 from the freeze epoch onward, treating the freeze as the single schedule event intrinsic to QAT, and halt training if any update produces a non-finite loss.

2.2. HAWQ-V2

A recent review by Rakka *et al.* (9) identifies HAWQ (Hessian AWARE Quantization) (4; 3; 10) among state-of-the-art mixed-precision approaches. In HAWQ-V2 (3), Dong *et al.* determine the relative sensitivity of layers via the Hessian trace, $\text{tr}(\mathbf{H}) = \text{tr}(\frac{\partial^2 \mathcal{L}}{\partial \theta^2})$, a measure of loss curvature. While exact computation is intractable for large models, this can be estimated using the Hutchinson algorithm (1) with m random probes (vectors) as $\text{tr}(\mathbf{H}) \approx \frac{1}{m} \sum_{i=1}^m \mathbf{z}_i^T \mathbf{H} \mathbf{z}_i$, where $\mathbf{z} \in \mathbb{R}^d$ is a random vector with $\mathbb{E}[\mathbf{z}\mathbf{z}^T] = \mathbf{I}$, so that $\text{tr}(\mathbf{H}) = \mathbb{E}[\mathbf{z}^T \mathbf{H} \mathbf{z}]$. (Gaussian $\mathcal{N}(0, 1)$ probes satisfy this; we follow `PyHessian` and use Rademacher ± 1 probes, which are unbiased for the same reason.)

Layer l ’s relative sensitivity at bit-width b is defined as $\Omega_l(b) = \frac{\text{tr}(\mathbf{H}_l) \|Q_b(\mathbf{W}_l) - \mathbf{W}_l\|_2^2}{\text{tr}(\mathbf{H})}$, with $\text{tr}(\mathbf{H})$ the trace normalized by layer size (number of elements) and $\|Q_b(\mathbf{W}_l) - \mathbf{W}_l\|_2^2$ the L_2 -norm of quantization error at bit-width b . HAWQ-V2 allocates bit-widths by minimizing $\sum_l \Omega_l(b_l)$ subject to a budget $\sum_l n_l b_l \leq B$, which can be formulated as an Integer Linear Programming (ILP) problem to find the Pareto-optimal quantization per layer. In this report we follow the authors (10) in using `PyHessian` to compute the trace and `PuLP` to solve the ILP; we minimally modified the `PyHessian` vendor repo to return mean trace *per layer* rather than a single scalar for the whole model. Allocation remains *per layer*: one bit-width per weight tensor. When we later combine HAWQ-V2 with per-channel quantization, only the affine scales (and thus the $\Omega_l(b)$ perturbation) become per-channel; the decision variables stay layer-wise. Although HAWQ-V2 was originally proposed for PTQ, its per-layer bit allocation can also serve as a fixed mixed-precision configuration. We therefore also explored using our HAWQ-V2 allocations under fold-aware QAT. For QAT, the allocation is computed once from the FP32 baseline and kept fixed throughout training, isolating the effect of retraining without re-optimizing the bit-width

assignment.

2.3. Learned Bit-Width (EdMIPS)

We also implement the differentiable search proposed by Cai and Vasconcelos (2). Each layer holds logits π over the candidate bit-widths (e.g. $\{2, 4, 8\}$) and quantizes to their learned combination $\bar{W} = \sum_l \pi_l Q_l(W_l)$, allowing bit-width to be optimized via gradient descent. In particular, we trained π with the Adam optimizer ($\beta = (0.5, 0.999)$, no weight decay).

For the complexity function f_c , we use the model size measured as the element-wise mean bits $f_c = \bar{b} = \sum_l n_l \mathbb{E}[b_l] / \sum_l n_l$ rather than the authors’ BOPS objective. This minimizes the edge-deployed model size in order to facilitate a more direct comparison against other quantization methods (which target bit-width of the model rather than BOPS) for this paper, specifically HAWQ-V2.

2.4. Frameworks and Code

We use `PyTorch` and `torchvision` for modeling, with `Hydra` for configuration and `wandb` for experiment tracking. We begin from `torchvision`’s ResNet-50, replacing the ImageNet stem (7×7 stride-2 convolution and max-pool) with a 3×3 stride-1 stem for CIFAR-100, so that 32×32 inputs are not prematurely downsampled (spatial resolution 32, 16, 8, 4 across stages). This yields 54 quantizable layers: 53 fused convolution–BatchNorm pairs and one unfolded fully-connected layer. The shared quantization components (affine quantize/dequantize, STE, range observers, and fake-quant wrappers) were implemented ourselves, and BatchNorm folding, the freeze-then-fold QAT modules, config builders, and the EdMIPS search were built on top of these components.

3. Experiments and Results

All models were trained on CIFAR-100’s 50k training set (10% reserved for validation) for 200 epochs using categorical cross-entropy loss, an SGD optimizer with 0.9 momentum, a cosine learning-rate schedule, and random seed 7643. Models were evaluated on CIFAR-100’s 10k test set.

PTQ starts from the FP32 baseline and folds BatchNorm before calibrating on 1,000 fixed training images; quantizer scale and zero-point are *not* learned in PTQ but are set by calibration statistics. QAT trains from scratch under identical hyperparameters and the same training recipe, and the weights are adapted *through* the quantizer via the STE. HAWQ-V2 starts from the same folded FP32 baseline as PTQ, estimating each layer’s average Hessian trace by Hutchinson’s method with 300 Rademacher probes over the same 1,000 calibration images before solving the ILP with `PuLP`. Weights are rounded once at the chosen widths without further fine-tuning (as in (3)). To evaluate the effect of retraining independently of bit allocation, we addi-

Quantization	b^w	top-1	top-5	Comp.	GBOPS
FP32 baseline	32	79.80	95.09	1.00×	1329.2
PTQ w8a32	8	79.63	95.14	3.99×	332.3
PTQ w8a8	8	79.71	95.14	3.99×	83.1
PTQ w4a32	4	8.39	22.95	7.94×	166.1
PTQ w4a4	4	7.12	20.02	7.94×	20.8
QAT w8a32	8	77.15	94.24	3.99×	332.3
QAT w8a8	8	76.99	94.06	3.99×	83.1
QAT w4a32	4	75.72	93.65	7.94×	166.1
QAT w4a4	4	66.89	90.05	7.94×	20.8
HAWQ-V2 a32	6.97	78.65	94.39	4.57×	316.2
HAWQ-V2 a32	5.99	78.87	94.40	5.31×	304.4
Mixed QAT a32	5.99	77.11	94.23	5.31×	304.4
HAWQ-V2 a32	4.99	71.73	90.55	6.37×	271.6
HAWQ-V2 a32*	4.00	3.89	12.90	7.95×	224.9

Table 1. Per-tensor ResNet-50 / CIFAR-100 top-1 and top-5 test accuracy. b^w is the element-weighted average weight width. Compression is measured relative to the folded FP32 baseline. HAWQ-V2 rows are weight-only (a32). *Per-tensor HAWQ-V2 collapsed at INT4 using $\{2, 4, 8\}$ bit-width options; subsequent HAWQ-V2 runs used only $\{4, 8\}$ bit-widths.

b^w	a	PTQ	QAT	QAT–PTQ	QAT–FP32
8	32	79.63	77.15	−2.48	−2.65
8	8	79.71	76.99	−2.72	−2.81
4	32	8.39	75.72	+ 67.33	−4.08
4	4	7.12	66.89	+ 59.77	−12.91

Table 2. Recovery of PTQ accuracy by QAT at matched bit-width and compression: the same configuration reached by training through the rounding operation (QAT) instead of applying it afterward (PTQ).

tionally reuse the selected 6-bit HAWQ-V2 allocation as a fixed layer-wise precision configuration for fold-aware QAT. The allocation is computed once from the FP32 baseline and held fixed throughout QAT training, so any performance differences reflect the optimization procedure rather than changes in bit allocation. As a gradient descent–based method, EdMIPS starts from the same baseline as QAT, learns an optimal allocation as training proceeds, and uses a softmax-based selection mechanism at the BatchNorm freeze point. For the remaining epochs, it is identical to fixed-allocation quantization at its selected bit width.

Compression is computed analytically as the ratio of parameter bits, taken over the parameter set after BatchNorm folding, relative to the folded FP32 baseline. We also report BOPS as a measure of arithmetic cost, which is defined as the number of multiply–accumulate (MAC) operations of the forward pass weighted by operand precision, $\sum_l \text{MACs}_l \cdot b_l^w \cdot b_a$, where b_l^w is the weight bit-width of layer l and b_a the activation bit-width; one GBOPS is 10^9 such operations.

As seen in Table 1, the FP32 baseline generalizes well

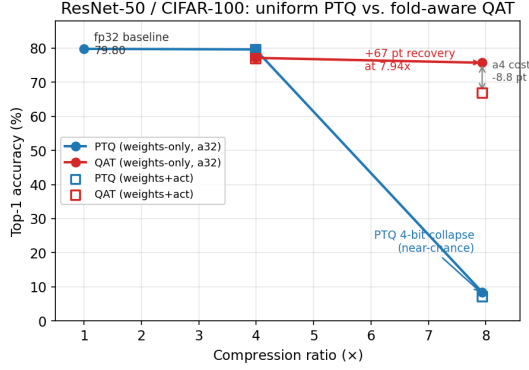


Figure 1. Top-1 vs. compression on CIFAR-100. PTQ collapses at INT4 (7.94 $\times$) while fold-aware QAT holds accuracy, a +67-point gap at identical compression. Open squares mark weights-plus-activation configurations; the -8.8-point drop at QAT INT4 is the activation-quantization cost.

(79.8% top-1; 95.09% top-5). At INT8, PTQ is lossless within noise (79.63% top-1 for weights-only, 79.71% with INT8 activations, both within 0.2 points of the baseline at 3.99 $\times$ compression). QAT at INT8 lands 2.7 points lower than weight-only quantization (77.15% and 76.99%). Because INT8 PTQ shows the quantization itself is nearly free, this gap is almost entirely the cost of training the QAT model from scratch through the freeze-then-fold schedule rather than a quantization penalty; at INT8, therefore, QAT incurs a fixed retraining cost with no offsetting benefit and is dominated by PTQ.

At INT4, this ordering reverses. Per-tensor PTQ collapses to near-chance (8.39% weights-only, 7.12% with activations, against a 1% random floor on 100 classes), because folding concentrates weight magnitude into a few channels and produces per-channel ranges too wide for one per-tensor 4-bit grid. Fold-aware QAT instead holds at 75.72% and 66.89% at the same 7.94 $\times$ compression, a recovery of 67.3 and 59.8 points, respectively (Table 2). This is a central result: the final 50 folded and quantized epochs let the optimizer reshape the folded weights to fit the 4-bit grid, which post-hoc calibration cannot do. (Section 3.2 shows that switching the *scale* granularity to per-channel is an alternative way past this cliff for PTQ, without retraining.) Figure 1 plots the top-1 accuracy of the two uniform arms against their compression ratio.

The activation-quantization cost is isolated by comparing the weights-only (a32) and weights-plus-activation (a8, a4) configurations at fixed weight width. At 8-bit weights, the loss in top-1 accuracy is negligible (0.16 points for QAT). At 4-bit weights, the loss grows to 8.8 points in QAT (from 75.72% to 66.89%); in PTQ, accuracy has already collapsed at INT4 weights, so the effect is masked. Low-precision activation quantization becomes the dominant error source only once the weights themselves are functional.

The fold-aware schedule is visible in the diagnostics (Figure 2). Loss descends normally through the live-BatchNorm phase; at the freeze (epoch 150) the tensor under the quantizer switches from the raw convolution weight to the folded weight, and the mean STE-hidden rounding error mean $|w - \text{dequant}(\text{quant}(w))|$ jumps (from 3.9×10^{-4} to 1.1×10^{-3} at 8-bit, and from 5.9×10^{-3} to 1.3×10^{-2} at 4-bit), then decays over the final 50 epochs as the weights adapt to their folded, quantized form. The 4-bit error is roughly an order of magnitude larger throughout, consistent with the coarser grid, yet the saturated fraction (weights clipped at $q_{\min}/q_{\max}$) holds at 0% for both runs: the recovery comes from redistributing weights within the grid, not from clipping. The 4-bit-activation arm shows a larger train-to-deploy gap because activations are calibrated post-hoc rather than trained through. Validation loss rises slightly after the freeze as the learning rate anneals and training loss keeps falling, a mild overfitting signature; we select the final epoch deliberately, since only a post-freeze checkpoint deploys the folded model that is evaluated.

3.1. Mixed-Precision Results

As shown in Table 1, HAWQ-V2 with per-tensor quantization was not able to resolve the INT4 cliff, as at 4.0 average bits per weight the accuracy collapses to 3.89%. However, at a budget of 6.0 bits per weight, HAWQ-V2 reaches 78.87% at 5.31 $\times$ compression, only 0.93 points below the FP32 baseline and within 0.9 points of INT8 PTQ while storing a third less. None of the uniform post-training configurations reach this operating point. Widening the budget to 7.0 mean bits does not help (78.65% at 4.57 $\times$), while tightening it to 5.0 costs 7.1 points (71.73% at 6.37 $\times$).

This INT4 collapse was the only per-tensor run whose candidate set included 2-bit precision, and 6 of the 54 layers returned a *non-positive* average Hessian trace, which makes a layer look “free” to the objective so that each such layer was allocated a bit-width of 2. This result is worse than not allocating at all: uniform PTQ at the equivalent compression (w4a32 at 7.94 $\times$) achieves a higher accuracy of 8.39%. Restricting the candidate set to {4, 8} for the remaining HAWQ-V2 runs recovered accuracy.

When evaluated using per-channel quantization (discussed in more detail below), both HAWQ-V2 and EdMIPS were given the same candidate set {2, 4, 8}, 4-bit width target, and hyperparameters as mentioned earlier, and arrive at relatively similar accuracies for 32-bit activations. However, HAWQ-V2 is more robust to the INT4 activation quantization than EdMIPS (70.70% vs 66.59% top-1 accuracy, respectively). Inspecting the allocations reveals that HAWQ-V2 has a distribution of 2/29/23 (2 layers at two bits, 29 layers at four bits, 23 layers at eight bits) whereas EdMIPS converged to 21/15/18. This allocation increases the number of layers at 2 bits and thus explains the reduced

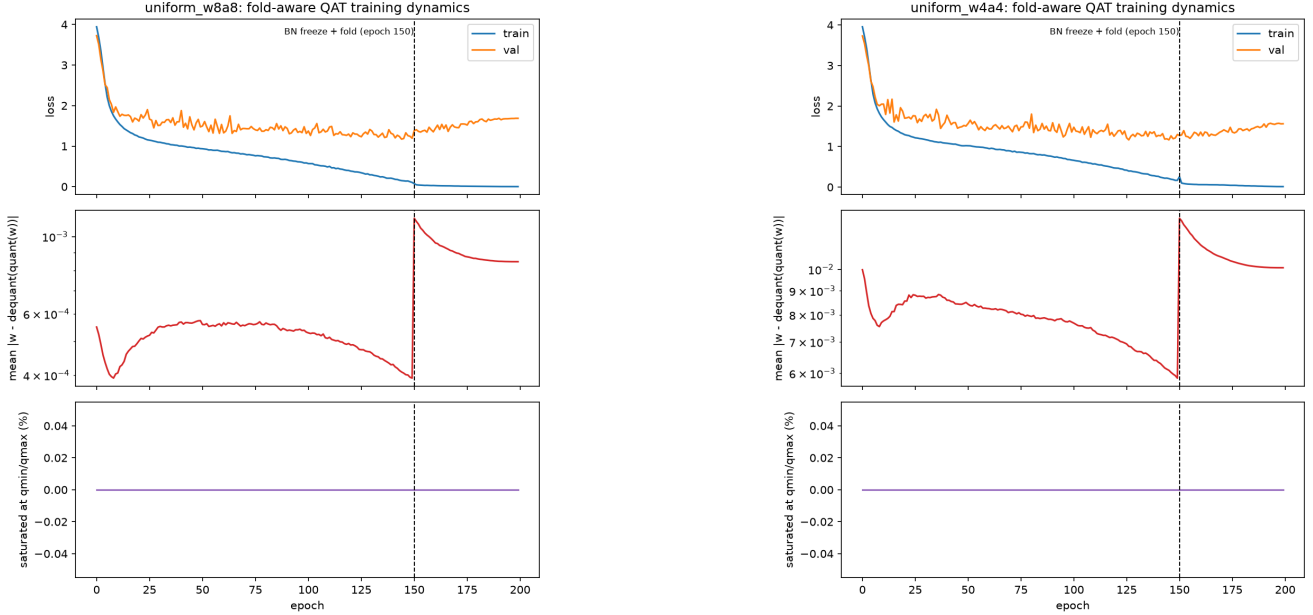


Figure 2. Fold-aware QAT dynamics (*left*, INT8; *right*, INT4). (*Top*) train/val loss with the BatchNorm freeze and fold at epoch 150. (*Middle*) STE-hidden rounding error, which jumps at the fold then decays. (*Bottom*) Saturated fraction, which is flat at 0%.

accuracy of the model at lower precisions.

We also observe that HAWQ-V2 allocates in an interpretable way: `layer1` convolutions and the linear layer at 8 bits, `layer3` predominantly at 4, and the two 2-bit allocations being in `layer4`. This supports the general intuition that CNNs learn local, smaller, and high-curvature layers earlier, with deeper layers being more global and robust to rounding. EdMIPS, by contrast, does not have a similarly interpretable allocation strategy.

We further evaluate the same HAWQ-V2 bit allocation (candidate set $\{4, 8\}$, target average 6 bits) under two regimes to compare the performance of mixed-precision PTQ and mixed-precision QAT. In the PTQ setting the per-layer widths are assigned once to the converged FP32 model and evaluated directly. In the QAT setting the identical allocation is used as the fixed per-layer weight precision inside the fold-aware schedule: BatchNorm remains live for the first 150 of 200 epochs, after which the statistics are frozen and the convolutions are folded; weight fake-quantization is applied throughout training while activations stay in FP32 and are calibrated only at evaluation. The HAWQ-V2 allocator produces an element-weighted average precision of 5.99 bits, corresponding to a $5.31\times$ reduction in model size (17.7 MB). Under this allocation the mixed-precision PTQ model reaches 78.87% top-1 / 94.40% top-5, while the corresponding mixed-precision QAT model reaches 77.11% top-1 / 94.23% top-5 at the same footprint. The 1.76 percentage point reduction in top-1 accuracy indicates that the HAWQ-V2 allocation already provides an effective mixed-precision solution under the current 6-bit

budget, with limited opportunity for further improvement through QAT. Nevertheless, the experiment demonstrates that Hessian-guided mixed-precision bit allocations can be directly integrated into a fold-aware QAT pipeline without additional bit-width optimization, producing a deployable mixed-precision model that preserves the $5.31\times$ storage reduction achieved by PTQ.

3.2. Per-Channel Quantization Results

As shown in Table 3, per-channel quantization allows not only INT4 PTQ to avoid collapse but also QAT as low as INT2. Per-tensor PTQ w4a32 achieved only 8.39% top-1, whereas the same configuration with per-channel scales reaches 78.33%. Note that the per-channel grid was re-run (including a separate FP32 baseline at 80.17%), so absolute numbers are not bit-identical to Table 1; the comparison of interest is within this table. Figure 3 summarizes both per-tensor and per-channel grids on a shared accuracy-compression plot.

However, per-channel quantization requires a scale tensor per output channel, thus increasing storage slightly. Hence the per-channel PTQ w4a32 model achieves $7.87\times$ compression versus $7.94\times$ for its per-tensor counterpart.

Similar results hold for the mixed-precision arms. With per-channel scales, HAWQ-V2 at $\bar{b} \approx 4$ no longer collapses (77.35% a32) and remains above EdMIPS under a4 (70.70% vs. 66.59%). These HAWQ rows still use *one bit-width per layer*; “per-channel HAWQ-V2” means the ILP’s $\Omega_l(b)$ and the fake-quantizers use per-channel scales, not that bits are assigned per output channel.

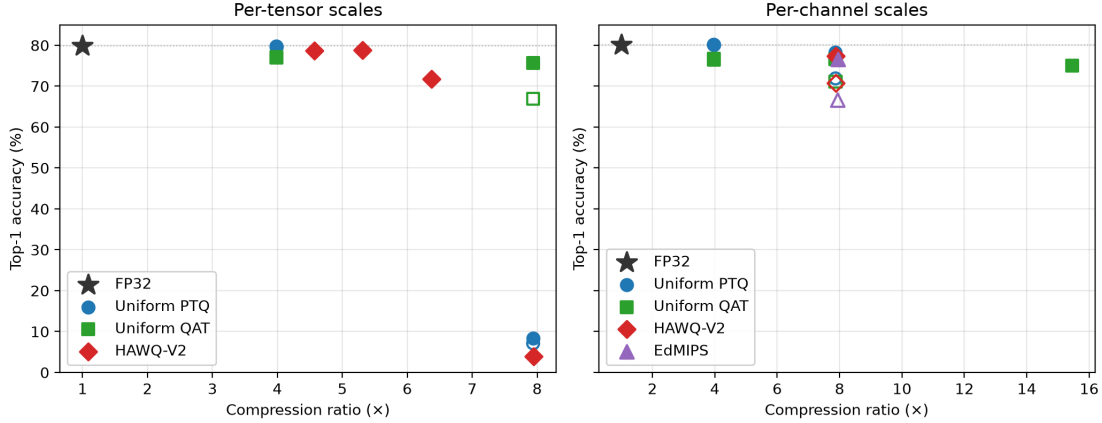


Figure 3. Top-1 vs. compression across all reported techniques. (*Left*) Per-tensor scales (Table 1): uniform PTQ collapses near $8\times$ while fold-aware QAT holds, and HAWQ-V2 PTQ fills intermediate budgets before falling off when 2-bit is admitted. (*Right*) Per-channel scales (Table 3): the INT4 cliff disappears for PTQ; HAWQ-V2 and EdMIPS sit near $8\times$ with EdMIPS more sensitive to a4. Filled markers are weights-only (a32); open markers include activation quantization. Each panel uses its own FP32 baseline for the compression ratio.

Config	$\bar{b}$	top-1	top-5	Comp.	GBOPS
FP32 baseline	32	80.17	95.07	1.00×	1329.2
PTQ w8a32	8	80.12	95.05	3.97×	332.3
PTQ w8a8	8	80.14	95.08	3.97×	83.1
PTQ w4a32	4	78.33	93.97	7.87×	166.1
PTQ w4a4	4	71.90	90.03	7.87×	20.8
QAT baseline	32	76.79	94.17	1.00×	1329.2
QAT w8a32	8	76.74	94.48	3.97×	332.3
QAT w8a8	8	76.48	94.38	3.97×	83.1
QAT w4a32	4	76.63	94.08	7.87×	166.1
QAT w2a32	2	75.01	94.29	15.46×	83.1
HAWQ-V2 a32	3.99	77.35	94.03	7.89×	224.6
HAWQ-V2 a4	3.99	70.70	91.06	7.89×	28.1
EdMIPS a32	3.96	76.59	94.12	7.94×	184.7
EdMIPS a4	3.96	66.59	89.28	7.94×	23.1

Table 3. Per-channel quantization CIFAR-100 test accuracy. $\bar{b}$ is the element-weighted average weight width. Compression is measured relative to the folded FP32 baseline.

4. Discussion and Future Work

Our experiments show that the benefit of QAT depends on how lossy the underlying quantization is. Where quantization is nearly free, retraining recovers nothing that was lost, and cost is what remains. This is the case at INT8 and at per-tensor mixed-precision budgets of six average bits or more, where quantization is lossless within noise: PTQ dominates, and fold-aware QAT pays a fixed retraining penalty of ~ 2.7 top-1 points with no offsetting benefit. The exception is the per-tensor INT4 cliff, where PTQ collapses to near-chance because folding concentrates weight magnitude into a few channels and produces per-channel ranges too wide for one per-tensor grid. This cliff has two potential solutions. The first is to train through the rounding

operation, which recovers 67 points over per-tensor PTQ. The second is to switch the scale granularity to per-channel, which reaches 78.33% weights-only without retraining at all. Since per-channel scaling also lifts PTQ past the cliff, the distinct value of fold-aware QAT lies in the operating points that post-hoc calibration cannot reach, most notably 2-bit weights at 75.01%. For mixed precision, Hessian-guided HAWQ-V2 fills the intermediate budgets that neither uniform arm occupies, and its allocations are interpretable, holding the early, high-curvature layers at high precision. Its candidate set must be constrained however, as admitting a 2-bit candidate lets layers with non-positive estimated Hessian trace appear free to the objective, which collapses accuracy below even uniform PTQ at matched compression.

Several potential directions follow from these findings. Training through activation quantization, rather than calibrating it post-hoc, would close the 8.8-point INT4 activation gap observed under QAT. Extending the per-tensor fold-aware schedule to 2-bit weights would test whether trained quantization can hold the ultra-low-precision regime that per-channel scaling already reaches. Exporting genuine INT4 and INT8 kernels would replace our simulated FP32 fake-quantized cost with wall-clock latency on real hardware. Finally, mixed-precision QAT could be improved through tighter bit-width constraints, joint weight-activation quantization, and post-freeze learning-rate and gradient-clipping schedules tuned specifically for the mixed-precision setting.

5. Work Division

The team divided the work by quantization arm, as described in Table 4.

Student Name	Contributed Aspects	Details
Bo Ding	ResNet-50, training loop, mixed-precision QAT	Integrated ResNet-50 architecture and model registry support. Integrated, trained, evaluated, and debugged the mixed-precision QAT pipeline using HAWQ-V2-generated bit allocations, including QAT fine-tuning, validation, and performance analysis of mixed-bit quantized models.
J. Keith Hall	HAWQ-V2, core quantization primitives	Built Observer class quantization calibrator, quantize/dequantize primitives, straight-through estimator, min-max and percentile calibration with EMA smoothing, weight and activation fake quantization wrapper classes, HAWQ-V2 modification to <code>PyHessian</code> , and HAWQ-V2 allocator using <code>PuLP</code>
Dustin Newman	Scaffolding, EdMIPS, per-channel scaling	Initial project scaffolding. Implemented EdMIPS, softmax mixture quantizer, Adam logits optimizer, bit operations metric, and hinged complexity term over model size. Implemented per-channel quantization and ran on QAT, PTQ, HAWQ-V2, and EdMIPS families.
Schuyler R. Wang	Evaluation, uniform PTQ, fold-aware QAT	Built the evaluation harness (top-1/top-5, analytic compression, latency, plotting, results schema). Implemented uniform PTQ: BatchNorm folding, calibration infrastructure, observer fixes, and the eight- and four-bit grid with weights-only and weights-plus-activation variants. Implemented and debugged fold-aware QAT and produced the training-dynamics diagnostics.

Table 4. Contributions of team members.

References

- [1] Haim Avron and Sivan Toledo. Randomized algorithms for estimating the trace of an implicit symmetric positive semi-definite matrix. *J. ACM*, 58(2), Apr. 2011. [2](#)
- [2] Zhaowei Cai and Nuno Vasconcelos. Rethinking differentiable search for mixed-precision neural networks. In *CVPR*, 2020. [1](#), [3](#)
- [3] Zhen Dong, Zhewei Yao, Daiyaan Arfeen, Amir Gholami, Michael Mahoney, and Kurt Keutzer. Hawq-v2: Hessian aware trace-weighted quantization of neural networks. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 18518–18529. Curran Associates, Inc., 2020. [1](#), [2](#), [3](#)
- [4] Zhen Dong, Zhewei Yao, Amir Gholami, Michael Mahoney, and Kurt Keutzer. Hawq: Hessian aware quantization of neural networks with mixed-precision. In *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 293–302, 2019. [2](#)
- [5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *CoRR*, abs/1512.03385, 2015. [1](#)
- [6] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew G. Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. *CoRR*, abs/1712.05877, 2017. [2](#)
- [7] Raghuraman Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: A whitepaper. *CoRR*, abs/1806.08342, 2018. [1](#), [2](#)
- [8] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009. [1](#)
- [9] Mariam Rakka, Mohammed E. Fouda, Pramod Khargonekar, and Fadi Kurdahi. A review of state-of-the-art mixed-precision neural network frameworks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12):7793–7812, 2024. [2](#)
- [10] Zhewei Yao, Zhen Dong, Zhangcheng Zheng, Amir Gholami, Jiali Yu, Eric Tan, Leyuan Wang, Qijing Huang, Yida Wang, Michael Mahoney, and Kurt Keutzer. Hawq-v3: Dyadic neural network quantization. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 11875–11886. PMLR, 18–24 Jul 2021. [2](#)

Github Repo

<https://github.gatech.edu/dnewman49/cs-7643-group-project>